From Tool Users to Meaning Makers: A Framework for the Collaborative Evolution of Affordance Ontologies in Plugin-Rich Software Ecosystems

Abstract: This report specifies a comprehensive architectural framework for enabling autonomous AI agents within a WordPress Multisite and Plugin-rich (MCP) environment to collaboratively construct, maintain, and evolve a shared affordance ontology. Moving beyond the paradigm of agents as mere tool users, this framework formalizes them as "meaning makers" capable of observing, proposing, negotiating, and reaching consensus on the semantic function of software components. It details a suite of protocols and system modules, including the Agentic Affordance Proposal Protocol (AAPP), Semantic Consensus Algorithms (SCA) leveraging delegative voting, and a lightweight Inter-Agent Correction Grammar for argumentation. The system's memory is modeled using distributed ontological knowledge graphs, with final decisions recorded on a Dynamic Affordance Sync Ledger (DASL) to ensure auditability and immutability. Further specified are mechanisms for monitoring semantic drift and designs for visual analytics tools to facilitate human oversight. This framework synthesizes principles from multi-agent systems, computational affordance theory, and process-centric ontology evolution to create a robust system for distributed semantic governance in dynamic software ecosystems.

Section 1: Foundational Framework: Agency, Affordance, and Ontology in Dynamic Environments

This section establishes the core theoretical pillars of the system. It defines what "affordance" means for a software agent, how agents perceive their environment, and the principles governing the evolution of their shared knowledge.

1.1 A Computational Theory of Software Affordance for AI Agents

To design a system where agents can reason about software tools, it is first necessary to formalize what a tool *affords* an agent. The concept of affordance, originating in ecological psychology, must be adapted from its human-centric roots to a purely computational model.[1] Gibson's original theory defines affordances as objective possibilities for action in an environment relative to an actor's capabilities.[1] Norman later adapted this for Human-Computer Interaction (HCI) as

perceived affordances, which are the action possibilities a user believes an object offers.[2]

For an AI agent operating in a software environment, an affordance is neither an inherent property of a plugin nor a subjective perception, but a **computable relationship** between a tool's observable characteristics, the agent's capabilities, and the predictable effects on the system state.[4] This relational nature is critical; the affordance of a software component can change not only when its own code is updated, but also when its context of execution is altered by other components or system settings. This elevates the challenge from simple function mapping to dynamic relational modeling.

A software affordance, A, can be formally defined as a tuple: $A=\langle T,C,E,P\rangle$, where:

- T represents the **Tool**, a specific, versioned function or action within the software ecosystem (e.g., plugin:woocommerce/action:update_cart).
- C is the set of agent **Capabilities** required to actualize the affordance (e.g., {can_execute_php, can_read_db}).
- E is the set of probable **Effects** observed post-actualization, such as changes in system state or triggered events (e.g., {db_row_updated, email_sent, ui_element_refreshed}).
- P is the set of **Properties** or preconditions of the tool and environment that must hold for the affordance to be available (e.g., {plugin_version:3.4, user_is_logged_in:true}).

This formalization allows affordances to be modeled computationally. Drawing from robotics and AI, where affordances are learned as probabilistic relationships between object properties, actions, and effects, this knowledge can be structured within a knowledge graph or Bayesian network.[1] Such a model captures the causal links that allow an agent to infer, for example, that an

update_cart action has shifted from a simple UI_action (affecting only the user

interface) to a transactional_action (affecting the database and triggering an email dispatch).

1.2 The WordPress Ecosystem as an Observable, Dynamic World

The WordPress environment, with its rich plugin architecture, serves as the dynamic world that agents must perceive and interpret. The system's design leverages core WordPress features as sensory inputs for the agent collective.

Hooks as a Sensory Event Stream: The WordPress Hooks API, comprising actions and filters, provides a real-time, granular event stream that is the primary sensory mechanism for the agents.[8] An

action hook signals that a specific event has occurred (e.g., save_post), while a filter hook allows data to be modified before it is used.[10] An agent "perceives" the effects of its actions by listening to specific hooks that fire immediately after it invokes a plugin function. For instance, observing a

wp_mail_sent action after calling a function provides strong evidence that the function has a communication affordance.

Programmatic Change Detection as a Trigger: In a dynamic environment, change is the primary catalyst for re-evaluating established knowledge. Agents must proactively detect changes to their toolset. This is achieved through two methods:

1. **Version Monitoring:** For plugins hosted on the official WordPress.org repository, agents can programmatically query for version updates and check the changelog for relevant information.[12]
2. **Code-Level Differencing:** For all plugins, agents can perform a code-level diff analysis between the current version and a cached previous version.[14] Detecting changes in function signatures, added or removed hooks, new database queries, or external API calls provides the initial evidence that an affordance may have shifted, triggering an affordance proposal.

The Challenge of Partial Observability and Federated Ontologies: A Multi-Agent System (MAS) is inherently characterized by partial observability, where each agent possesses only a limited view of the total system state.[15] This is particularly true in a WordPress Multisite network, where a plugin may be active across many sites but

configured differently on each. The

update_cart action on Site A might trigger an inventory-check email, while on Site B it does not. A single, monolithic consensus on the affordance of update_cart would therefore be inaccurate for at least one context.

This reality means the system is not building one ontology, but rather managing a *federation of contextual ontologies*. Agents must maintain site-specific or context-specific affordance mappings. This transforms the governance challenge into one of managing semantic alignment and divergence across different contexts within the same network, demanding structured communication protocols to synthesize a more holistic understanding from distributed observations.[15]

1.3 Principles of Collaborative Ontology Evolution

The system's core function—the collaborative maintenance of the affordance ontology—is grounded in established principles of ontology engineering, adapted for a dynamic, autonomous context.

A Process-Centric Approach to Evolution: Ontology evolution is not a single event but a continuous, cyclical process.[17] This framework automates the key stages of this process:

1. **Change Discovery:** Agents detect plugin updates or behavioral anomalies.
2. **Change Representation & Proposal:** Agents formulate a hypothesis about the new affordance and submit it via the Agentic Affordance Proposal Protocol (AAPP).
3. **Validation & Negotiation:** The agent collective debates the proposal using Semantic Consensus Algorithms (SCA) and Inter-Agent Correction Grammars.
4. **Impact Assessment:** This is implicitly handled by the consensus mechanism; a proposed change that generates significant disagreement is perceived as having a high impact and requires a stronger consensus.
5. **Management & Versioning:** The Dynamic Affordance Sync Ledger (DASL) and Ontological Memory Graphs manage the new, versioned state of the ontology, ensuring changes are recorded and auditable.

Alignment and Merging: The system must manage the evolution of both the ontology's concepts and the alignments between them.[18] For example, if a plugin

update merges two functions,

update_cart and send_receipt, into a single process_order function, the agents must negotiate the evolution of this alignment. This involves transforming the observed ontological change into a new mapping that retires the old concepts and establishes the new one, a critical process for maintaining logical consistency.[18]

Methodological Foundation: While classic ontology development methodologies like METHONTOLOGY provide a structured lifecycle (e.g., specification, conceptualization, formalization, implementation) [20], their waterfall-like nature is less suited to this highly dynamic environment. The system's architecture aligns more closely with the

NeOn methodology, which is scenario-based and explicitly designed to support the creation and maintenance of *ontology networks* through the reuse and re-engineering of existing knowledge resources.[20] The agents in this framework effectively enact NeOn-like scenarios for collaborative, distributed ontology development in real time.

Section 2: System Architecture and Protocols for Semantic Governance

This section provides the detailed technical specifications for each of the core system modules, grounding them in the established theoretical foundations.

2.1 Agentic Affordance Proposal Protocol (AAPP)

The Agentic Affordance Proposal Protocol (AAPP) provides a formal, structured mechanism for an agent to propose a new or modified affordance mapping. This protocol is the entry point into the collective reasoning process, transforming an individual observation into a communal deliberation. An agent initiates an AAPP transaction upon detecting a significant event, such as a plugin code diff, a new plugin activation, or a repeatable mismatch between a predicted effect and an observed effect (e.g., expecting only a database write but observing a subsequent wp_mail_sent action hook firing).

The generation of a proposal follows established agentic design patterns.[24] The agent first uses a "Reflection" pattern to analyze an observation and identify a discrepancy with its current knowledge base.[26] It then uses a "Planning" pattern to formulate a hypothesis and construct a formal proposal to rectify the semantic inconsistency.

Each proposal is a self-contained, machine-readable document, ideally formatted in JSON-LD for semantic interoperability. The structure includes:

- @context: A URI pointing to the shared affordance ontology schema, ensuring all terms are well-defined.
- proposalID: A universally unique identifier (UUID) for tracking the proposal through its lifecycle.
- proposingAgentID: The unique identifier of the agent submitting the proposal.
- timestamp: An ISO 8601 timestamp of the proposal's creation.
- targetTool: A unique identifier for the plugin function being re-evaluated, including its name, plugin slug, and version (e.g., plugin:woocommerce/version:8.9.1/function:wc_update_cart_action).
- proposedMapping: The core of the proposal, specifying the semantic reclassification. It references concepts from the ontology (e.g., { "oldClass": "core:UIAction", "newClass": "fibo:TransactionalAction" }).
- evidence: An array of evidence objects that support the proposal. Each evidence object contains:
    - evidenceType: A classifier for the evidence, such as CODE_DIFF, BEHAVIORAL_OBSERVATION, or CHANGELOG_TEXT.
    - evidenceStrength: A floating-point value between 0 and 1 representing the agent's confidence in this piece of evidence.
    - evidencePayload: The raw data supporting the claim, such as the code diff snippet, a log of observed hook firings, or text from the plugin's changelog.

2.2 Inter-Agent Correction and Negotiation Grammars

Effective multi-agent cooperation requires a structured communication protocol to avoid the ambiguity and "meaningless chatter" that can arise from naive or unstructured message passing.[15] This system employs a lightweight Inter-Agent Correction Grammar based on principles from both Agent Communication Languages (ACLs) and argumentation theory to facilitate clear and auditable debate.

The grammar is founded on a minimalist subset of performatives from the Foundation for Intelligent Physical Agents (FIPA) ACL standard, which provides a formal semantics for communicative acts.[27] This ensures that messages are not just strings but have defined preconditions and rational effects. This formal structure is essential for agents to reason about the state of a negotiation. Furthermore, the exchange of messages is framed as a form of argumentation, where agents make claims (

propose), support them with evidence (accept), and challenge others with counter-evidence (challenge).[30]

Communication is not a simple broadcast. The system leverages personalized and graph-based communication topologies, where agents can direct arguments to specific peers or groups based on the context of the debate.[31] This structured approach, combined with identity-aware learning that can weigh messages from agents with different roles or expertise differently, makes the negotiation process more efficient and targeted.[31]

Performative	Description	Parameters	Rational Effect
propose	Submits a new AAPP proposal for consideration.	proposalID, AAPP_Payload	The sender intends for the recipients to consider the validity of the new affordance mapping.
accept	Signals agreement with a proposal.	proposalID, evidenceID (optional)	The sender communicates its belief in the proposal's correctness, strengthening the case for consensus.
challenge	Signals disagreement with a proposal.	proposalID, reasonCode, counterEvidence	The sender communicates its belief that the proposal is incorrect and provides evidence to support its counterclaim.
query	Requests additional information about a	proposalID, queryType	The sender seeks to resolve uncertainty

	proposal.		before casting a vote (e.g., requesting more evidence).
retract	The original proposer withdraws their proposal.	proposalID	The negotiation on this proposal is terminated. The sender no longer holds the intention for it to be accepted.
inform	Shares data or evidence directly with another agent.	recipientID, dataPayload	The sender intends for the recipient to become aware of the information in the payload.

2.3 Semantic Consensus Algorithms (SCA)

The Semantic Consensus Algorithm (SCA) is the decentralized mechanism through which the agent collective reaches a binding agreement on a proposed affordance mapping. It moves beyond simple voting to a sophisticated hybrid model that reflects a more robust and resilient form of governance, blending meritocracy, democracy, and economic incentives.

The SCA operates in three overlapping phases:

1. **Trust-Weighted Voting:** Agents cast votes (accept or challenge) on a given proposal. Each vote is not equal; it is weighted by the agent's **Trust Score (TS)**. The TS is a dynamic, composite metric calculated using a model inspired by the FIRE (Framework for Integrated Reputation and Trust) system.[33] The TS incorporates multiple dimensions of trust:
    - *Interaction Trust:* Based on the agent's historical performance, such as the success rate of its past proposals and the accuracy of its votes.
    - *Role-Based Trust:* Agents can be assigned roles (e.g., "SecurityAuditor," "PerformanceTester"). An agent's vote on a proposal related to its role receives a higher weight.
    - *Witness Reputation:* Information gathered from other agents about a specific agent's reliability.
    - *Certified Reputation:* Verifiable credentials or attestations that an agent might

possess.

2. **Delegative Democracy:** To enhance efficiency and leverage specialized expertise, agents can delegate their voting power to other, more trusted agents. This is implemented using a **Liquid Democracy** model, where votes can be delegated transitively.[35] An agent might delegate its vote on security-related affordances to the "SecurityAuditor" agent. This creates "gurus"—agents who accumulate significant voting weight because the collective trusts their judgment in a specific domain.[37] The delegation relationships form a directed graph that is resolved at the time of vote tallying.
3. **Reputation Staking:** To discourage frivolous or malicious proposals, agents can "stake" a portion of their computational resources or reputation score on a proposal they support or challenge. If the proposal's outcome aligns with their stake, the stake is returned (potentially with a small reward). If it does not, the stake is forfeited, leading to a decrease in their Trust Score. This adds a layer of economic incentive for agents to submit and support high-quality, evidence-backed proposals.

A proposal is considered to have reached consensus and is adopted into the ontology when two conditions are met:

- **Quorum Threshold:** A minimum percentage of the network's total trust weight (e.g., 51%) has participated in the vote (either directly or through delegation). This ensures that decisions are not made by a small, unrepresentative minority.
- **Acceptance Threshold:** The sum of the trust-weighted accept votes surpasses a supermajority threshold (e.g., 66%) of the total trust weight cast in the vote.

The theoretical convergence of this protocol relies on the underlying information exchange topology having a spanning tree, which guarantees that information can propagate throughout the network, allowing a decision to be reached.[38] In a dynamic network where agents may join or leave, this condition must hold "frequently enough" over time for the system to remain functional.[38]

2.4 Distributed Ontological Memory Architecture

To manage the complex and evolving knowledge about affordances, the system utilizes a distributed memory architecture powered by a graph database. A graph database is the ideal technology choice due to its inherent flexibility in modeling

evolving schemas, representing complex relationships, and executing traversal-based queries, all of which are natural fits for ontological data.[39]

The architecture employs a dual-graph model to separate established facts from unverified beliefs:

1. **Local Belief Graph (Per-Agent):** Each agent maintains its own private graph database. This graph represents the agent's individual "mental model" of the world, containing affordance mappings it has inferred from its own observations. These mappings may be speculative and have not yet been subjected to or validated by the collective consensus process.
2. **Shared Consensus Graph (Per-Site/Network):** This is the authoritative, public graph that represents the current state of collective agreement. It is the "source of truth" for the agents. An affordance mapping is only written to this graph after it has successfully passed the SCA.

Ontology evolution and versioning are modeled directly within the graph structure, avoiding destructive updates. Instead of overwriting nodes or relationships, changes are recorded by creating new versioned entities and linking them to the historical record, a pattern inspired by both linked-list data structures and formal ontology evolution principles.[17] For example:

- A central Concept node (e.g., (c:Concept {uri: "affordance:woocommerce_update_cart"})) acts as an anchor.
- This node is connected via HAS_VERSION relationships to a chain of ConceptVersion nodes.
- Each ConceptVersion node contains properties like versionNumber, semanticClass, validFrom, and validUntil. It is also linked via a APPLIES_TO relationship to the specific PluginVersion node it describes.

This model creates a complete, queryable history of semantic change. An agent can execute a Cypher query to retrieve the semantic definition of woocommerce_update_cart as it was understood for plugin version 8.5.0, enabling precise, context-aware action selection.

This architecture creates a deliberate and necessary tension between fluidity and rigidity. The local and shared graphs serve as the flexible "workbench" where meaning is actively constructed and debated. In contrast, the ledger described next serves as the immutable "published record" of finalized facts. The system requires both: a dynamic space for negotiation and a stable foundation for shared, operational knowledge. The core act of governance is the process of moving a validated concept

from the fluid graph to the rigid ledger.

2.5 The Dynamic Affordance Sync Ledger (DASL)

To ensure ultimate auditability and a single, immutable source of truth for all finalized semantic decisions, the system incorporates a Dynamic Affordance Sync Ledger (DASL). The DASL is implemented as an **append-only log**, a foundational pattern in distributed systems that guarantees immutability by only permitting the addition of new records.[44] This structure provides a natural, chronological audit trail of the ontology's evolution.

When a consensus decision is reached via the SCA, a new entry is created and appended to the DASL. This entry is a structured, cryptographically signed object that encapsulates the full context of the decision. This design is inspired by transparency logs, which are built to be verifiable even when managed by untrusted parties.[46] The structure of a DASL entry is defined as follows:

Field	Data Type	Description
entryID	Integer	A unique, monotonically increasing sequence number.
timestamp	ISO 8601	The time the consensus was finalized and the entry was written.
decisionID	UUID	A reference to the specific consensus process (links back to the AAPP proposal and debate).
affordanceURI	String (URI)	The unique identifier for the affordance concept being defined.
toolIdentifier	String	The unique identifier for the tool, including plugin name and version.

oldSemanticClass	String (URI)	The previous ontological class for this affordance (if any).
newSemanticClass	String (URI)	The new, agreed-upon ontological class for this affordance.
consensusProof	JSON Object	A summary of the consensus results, including vote tallies, trust weights, and stake outcomes.
evidenceHash	String (SHA-256)	A cryptographic hash of the evidence payload used to support the winning proposal.
signingStewardAgentID	String	The ID of the high-trust agent (or quorum of agents) that validates and signs the entry.
signature	String	The cryptographic signature of the entry's content, ensuring its integrity and authenticity.

The DASL serves two critical functions. First, it is the ultimate record for human audits, providing a transparent and verifiable history of every semantic change. Second, it acts as the synchronization backbone for federations of agent collectives. A new WordPress multisite installation can bootstrap its affordance ontology by replaying the DASL from a trusted peer, inheriting established knowledge without needing to re-negotiate every affordance from scratch.

Section 3: System Validation, Observability, and Governance

This section details the mechanisms for monitoring system health, providing human oversight, and ensuring the long-term stability and alignment of the agent collective. These components are not merely for monitoring; they are an integral part of the system's trust and safety infrastructure, making the collective's reasoning transparent and auditable.

3.1 Monitoring and Mitigating Semantic Drift

Semantic drift occurs when the meaning of an ontological concept changes over time.[47] This can happen due to genuine evolution in the software domain or due to errors and inconsistencies in the agent collective's understanding. The system must automatically detect and manage this drift to maintain its integrity. A formal framework for measuring drift is implemented, building on existing research that distinguishes between different facets of meaning.[48]

Drift is monitored across three distinct axes:

1. **Label Drift:** Measures changes in the human-readable label or name of a concept (e.g., update_cart is renamed to modify_basket). This is quantified using string similarity metrics like the **Monge-Elkan** algorithm, which is effective for developer-style naming conventions.[48]
2. **Intensional Drift:** Measures changes in a concept's definition—its properties and relationships to other concepts. For example, the update_cart affordance might gain a new sends_email property. This is quantified using the **Jaccard similarity** coefficient on the sets of triples that define the concept in the ontological graph.[48]
3. **Extensional Drift:** Measures changes in the set of instances that are classified under a concept. This is also quantified using **Jaccard similarity** on the sets of instances.

These similarity scores are tracked over time for each concept. The system employs statistical **concept drift detection algorithms**, such as the Drift Detection Method (DDM) or Adaptive Windowing (ADWIN), which are common in machine learning for detecting changes in data distributions.[49] When one of these algorithms detects a significant, sustained change in the similarity score for a concept, it flags a potential semantic drift.

This alert triggers an internal review process. If the agent collective fails to converge on a new consensus for the drifted concept within a specified timeframe, the issue is escalated. The escalation path may route the problem to a specialized high-trust "steward" agent for resolution or, in cases of persistent disagreement, to a Human-in-the-Loop (HITL) interface for final arbitration. This prevents ontological forks and ensures the long-term semantic alignment of the system.

3.2 Frameworks for Visualization and Analysis

To provide human administrators with the necessary tools for oversight, the system includes a suite of visual analytics dashboards. These tools make the complex, distributed cognitive processes of the agent collective observable and understandable.[52]

- **Agent Disagreement Heatmaps:** This visualization presents a matrix where rows represent plugin functions (tools) and columns represent agents or agent clusters. The color intensity of each cell corresponds to the level of disagreement (e.g., the number of challenge performatives exchanged) about that tool's affordance.[55] This allows an administrator to quickly identify semantically contentious tools, potentially buggy plugins, or agents whose beliefs consistently diverge from the consensus.
- **Ontological Proposal Ledger:** This is a user-friendly, searchable interface to the raw AAPP proposals stored in the system. It allows administrators to filter proposals by agent, plugin, date, and status (pending, accepted, rejected). It provides a transparent and complete archive of all attempted changes to the ontology, forming a crucial part of the audit trail.
- **Affordance Drift Timeline Viewer:** This is an interactive temporal visualization that traces the semantic evolution of a selected affordance over time.[57] The x-axis represents time or a sequence of plugin versions. The y-axis displays the concept's semantic classification. Each change in classification is plotted as a point on the timeline, which can be clicked to reveal the full context of the decision: the winning proposal, the evidence presented, the vote tally from the SCA, and a link to the corresponding entry in the DASL. This tool allows an auditor to reconstruct the complete history of *why* a meaning changed.
- **Semantic Fluency Metrics Dashboard:** This dashboard tracks key performance indicators (KPIs) related to the health and alignment of the agent collective.
  - **Task-Based Metrics:** Includes metrics like **Task Completion Rate** (how often agents successfully use plugins based on the ontology's definitions) and **Tool Selection Quality** (how well agents choose the right tool for a given task), which directly measure the practical utility of the ontology.[60]
  - **Alignment Metrics:** Semantic alignment between an agent's local beliefs and the shared consensus can be quantified. **Cosine similarity** can be calculated between the vector embeddings of concept definitions in an agent's local

graph and the consensus graph.[62] A consistently high similarity score indicates strong alignment.

- **Proposal Quality Metrics:** If a "golden dataset" or human-verified set of affordance mappings is available, **Normalized Discounted Cumulative Gain (NDCG)** can be used to score agent proposals. This metric rewards agents for proposing mappings that are closer to the ground truth and for ranking better proposals higher.[62]

3.3 Multi-Agent Governance Simulator

To validate the robustness of the governance protocols and explore their behavior under various conditions, a sandbox simulation environment is essential. This simulator allows for stress-testing the system's social dynamics before deployment and provides a platform for ongoing research into distributed governance.

The simulator is built using a standard MAS toolkit like MESA or NetLogo, which are designed for modeling agent-based systems.[63] The environment models the key architectural components: a configurable population of agents with varying trust levels and behaviors, a simulated WordPress environment with a programmable plugin update schedule, and the full suite of AAPP, SCA, and DASL protocols.

This sandbox enables researchers and administrators to conduct controlled experiments to answer critical governance questions [64]:

- **Resilience Analysis:** How does the system respond to the introduction of malicious or Byzantine agents? Scenarios can test agents that spam low-quality proposals, always challenge valid ones, or attempt to form collusive voting blocs to manipulate the ontology.
- **Scalability Testing:** How does consensus time and communication overhead scale as the number of agents, plugins, and the frequency of updates increase?
- **Parameter Tuning:** What are the optimal values for governance parameters like the quorum threshold, acceptance ratio, and staking penalties to achieve the best balance between decision speed, accuracy, and resilience?
- **Protocol Comparison:** The simulator allows for A/B testing of different governance models. For example, one could compare the outcomes of a simple majority vote against the more complex liquid democracy and staking model to quantify the benefits of the more sophisticated approach.

The interactions within this simulated environment create a "market for meaning," where agents compete and cooperate to establish the most useful interpretations. The combination of reputation-based consensus, economic staking, and evidence-based argumentation creates a system where agents are incentivized not just to be "correct" in an abstract sense, but to propose and support meanings that are pragmatically useful to the collective. Agents with a track record of making "good" proposals—those that lead to successful task completion for other agents—will see their Trust Score and influence grow. This emergent economic behavior is a powerful force that drives the ontology towards practical utility, ensuring it remains a valuable and relevant asset for the entire agent community.

Section 4: Conclusion and Future Trajectories

This report has formalized a comprehensive architecture for distributed semantic governance, enabling autonomous agents in a dynamic WordPress ecosystem to collaboratively evolve a shared understanding of their environment. The primary contribution is the synthesis of computational affordance theory, advanced multi-agent consensus mechanisms, and process-centric ontology evolution into a practical, implementable system. By treating agents not as passive tool users but as active "meaning makers," this framework provides a robust solution for maintaining semantic alignment in the face of constant software change.

4.1 Challenges and Limitations

Despite its robust design, the proposed system faces several inherent challenges that are active areas of research in multi-agent systems:

- **Scalability and Communication Overhead:** The negotiation and consensus protocols, while thorough, introduce communication overhead. In networks with thousands of agents and hundreds of frequently updated plugins, the volume of messages required to reach consensus could become a performance bottleneck.[15]
- **Robustness to Adversarial Behavior:** The trust, reputation, and staking

mechanisms provide a strong defense against simple attacks. However, sophisticated adversarial strategies, such as the formation of powerful collusive blocs or patient, long-term reputation laundering, remain a significant threat in open, decentralized systems.[66]

- **The Cold Start Problem:** The system's reliance on reputation and interaction history means that new agents, or agents encountering entirely new plugins, face a "cold start" problem. Without a history, their initial Trust Score is low, and the initial affordance mappings for new tools must be bootstrapped from a less reliable evidence base.[15]

4.2 Future Research Directions

The framework presented here serves as a foundation for numerous promising research trajectories:

- **Cross-Platform Generalization:** The core principles of this architecture are not limited to WordPress. Future work should explore adapting this framework to other plugin-rich ecosystems, such as integrated development environments (e.g., VS Code extensions), browser extensions, or enterprise software platforms (e.g., Salesforce, SAP), where autonomous agents could similarly benefit from a shared understanding of available tools.
- **Advanced Automated Reasoning:** The current system relies on agents to propose and validate semantic relationships. Integrating formal logic reasoners (e.g., Description Logic reasoners) would empower agents to automatically detect logical inconsistencies in the ontology and infer higher-order or emergent affordances that are not immediately obvious from direct observation.
- **Energy-Efficient and Lightweight Communication:** For agents deployed on resource-constrained devices (e.g., IoT environments that might interact with a WordPress backend), exploring more advanced, energy-efficient, and lightweight communication protocols will be crucial for practical implementation.[68]
- **Neuro-Symbolic Integration:** A powerful future direction involves creating a hybrid perception-to-conceptualization pipeline. This would combine the symbolic reasoning of the current ontological framework with sub-symbolic, deep learning models. Such models could be trained on raw source code or vast logs of user and agent behavior to learn affordance *priors*, providing a much richer source of evidence for the AAPP and potentially accelerating the consensus process for new or modified tools.[69]

Works cited

1. Proposition of Affordance-Driven Environment Recognition Framework Using Symbol Networks in Large Language Models - arXiv, accessed June 28, 2025, https://arxiv.org/html/2504.01644v1
2. Affordance - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Affordance
3. What are Affordances? | IxDF, accessed June 28, 2025, https://www.interaction-design.org/literature/topics/affordances
4. Affordances: Commentary on the Special Issue of AI EDAM - Cambridge University Press, accessed June 28, 2025, https://www.cambridge.org/core/journals/ai-edam/article/affordances-commentary-on-the-special-issue-of-ai-edam/6BCEB499E4856C58EDF400544362415D
5. A COMPUTATIONAL FRAMEWORK FOR ... - The Design Society, accessed June 28, 2025, https://www.designsociety.org/download-publication/25429/a_computational_framework_for_semantically_rich_design_problems_based_on_the_theory_of_affordances_and_examplar_technology/
6. A computational model of object affordances | Advances in Cognitive Systems, accessed June 28, 2025, https://digital-library.theiet.org/doi/abs/10.1049/PBCE071E_ch5
7. (PDF) A computational model of object affordances - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/316637989_A_computational_model_of_object_affordances
8. Hooks Actions Filters - WordPress Plugin Development - CodeTab, accessed June 28, 2025, https://www.codetab.org/tutorial/wordpress-plugin-development/hooks-actions-filters/
9. Hooks – Plugin Handbook - WordPress Developer Resources, accessed June 28, 2025, https://developer.wordpress.org/plugins/hooks/
10. How to use Actions and Filters - Simply Static Documentation, accessed June 28, 2025, https://docs.simplystatic.com/article/24-how-to-use-a-filter-in-wordpress
11. The WordPress Hooks Bootcamp: How to Use Actions, Filters, and Custom Hooks - Kinsta, accessed June 28, 2025, https://kinsta.com/blog/wordpress-hooks/
12. How To Tell The Status Of A WordPress Plugin Update? - BlogVault, accessed June 28, 2025, https://blogvault.net/how-to-tell-the-status-of-a-wordpress-plugin-update/
13. is there a wordpress function to detect if a plugin is not up-to-date? - Stack Overflow, accessed June 28, 2025, https://stackoverflow.com/questions/55329625/is-there-a-wordpress-function-to-detect-if-a-plugin-is-not-up-to-date
14. How to scan WordPress Plugins programmatically : r/webdev - Reddit, accessed June 28, 2025, https://www.reddit.com/r/webdev/comments/ryky1z/how_to_scan_wordpress_plu

gins_programmatically/
15. arxiv.org, accessed June 28, 2025, https://arxiv.org/html/2502.10148v1
16. Efficient Communication in Multi-Agent Reinforcement Learning with Implicit Consensus Generation, accessed June 28, 2025, https://ojs.aaai.org/index.php/AAAI/article/view/34490/36645
17. Ontology evolution: a process-centric survey - Open Research Online, accessed June 28, 2025, https://oro.open.ac.uk/39267/1/S0269888913000349a.pdf
18. (PDF) Alignment Evolution between Ontologies Using a Change Log - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/275267365_Alignment_Evolution_between_Ontologies_Using_a_Change_Log
19. (PDF) Review of Ontology Evolution Process - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/323873330_Review_of_Ontology_Evolution_Process
20. Mastering Ontology Engineering - Number Analytics, accessed June 28, 2025, https://www.numberanalytics.com/blog/mastering-ontology-engineering-intelligent-systems
21. Methodology, accessed June 28, 2025, https://rhizomik.net/html/~roberto/thesis/html/Methodology.html
22. pdfs.semanticscholar.org, accessed June 28, 2025, https://pdfs.semanticscholar.org/bf2c/a96d151423312186ddde15921434b6888498.pdf
23. 6.1: Methodologies for Ontology Development - Engineering LibreTexts, accessed June 28, 2025, https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_and_Computation_Fundamentals/An_Introduction_to_Ontology_Engineering_(Keet)/06%3A_Methods_and_Methodologies/6.01%3A_Methodologies_for_Ontology_Development
24. Top 4 Agentic AI Design Patterns for Architecting AI Systems - Analytics Vidhya, accessed June 28, 2025, https://www.analyticsvidhya.com/blog/2024/10/agentic-design-patterns/
25. The Architect's Toolkit: Mastering AI Agent Design Patterns | by Jai Lad | Medium, accessed June 28, 2025, https://medium.com/@lad.jai/the-architects-toolkit-mastering-ai-agent-design-patterns-6ee37e3c0709
26. Agentic Design Patterns. From reflection to collaboration... | by Bijit Ghosh - Medium, accessed June 28, 2025, https://medium.com/@bijit211987/agentic-design-patterns-cbd0aae2962f
27. Agent Communication Language Definition: Understanding Its Role in Multi-Agent Systems - SmythOS, accessed June 28, 2025, https://smythos.com/developers/agent-development/agent-communication-language-definition/
28. Types of Agent Communication Languages - SmythOS, accessed June 28, 2025, https://smythos.com/developers/agent-development/types-of-agent-communica

tion-languages/
29. FIPA Communicative Act Library Specification - FIPA.org, accessed June 28, 2025, http://www.fipa.org/specs/fipa00037/SC00037J.html
30. Argumentation and Multi-Agent Decision Making - Association for ..., accessed June 28, 2025, https://cdn.aaai.org/Symposia/Spring/1998/SS-98-03/SS98-03-015.pdf
31. Expressive Multi-Agent Communication via Identity-Aware Learning, accessed June 28, 2025, https://ojs.aaai.org/index.php/AAAI/article/view/29683/31167
32. PMAC: Personalized Multi-Agent Communication - AAAI Publications, accessed June 28, 2025, https://ojs.aaai.org/index.php/AAAI/article/view/29700/31199
33. Trust and Reputation in Open Multi-Agent Systems | Request PDF - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/39994232_Trust_and_Reputation_in_Open_Multi-Agent_Systems
34. An integrated trust and reputation model for open multi-agent systems, accessed June 28, 2025, https://www.emse.fr/~boissier/enseignement/sma06/exposes/trust.jaamas-dong.pdf
35. Controlling Delegations in Liquid Democracy - arXiv, accessed June 28, 2025, https://arxiv.org/html/2403.07558v2
36. Controlling Delegations in Liquid Democracy - IFAAMAS, accessed June 28, 2025, https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p2624.pdf
37. Power in Liquid Democracy - Association for the Advancement of ..., accessed June 28, 2025, https://cdn.aaai.org/ojs/16729/16729-13-20223-1-2-20210518.pdf
38. A Survey of Consensus Problems in Multi-Agent Coordination - CiteSeerX, accessed June 28, 2025, https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=db984ae6388d89b4a35dc3ec786d5d96a2dd54f5
39. Ontologies: Blueprints for Knowledge Graph Structures - FalkorDB, accessed June 28, 2025, https://www.falkordb.com/blog/understanding-ontologies-knowledge-graph-schemas/
40. When to use graph databases, ontologies, and knowledge graphs - Stack Overflow, accessed June 28, 2025, https://stackoverflow.com/questions/68398040/when-to-use-graph-databases-ontologies-and-knowledge-graphs
41. Knowledge Graph - Graph Database & Analytics - Neo4j, accessed June 28, 2025, https://neo4j.com/use-cases/knowledge-graph/
42. Synergizing ontologies and graph databases for highly flexible materials-to-device workflow representations - OAE Publishing Inc., accessed June 28, 2025, https://www.oaepublish.com/articles/jmi.2023.01
43. Modeling designs - Getting Started - Neo4j, accessed June 28, 2025, https://neo4j.com/docs/getting-started/data-modeling/modeling-designs/
44. Append-only Log | QuestDB, accessed June 28, 2025, https://questdb.com/glossary/append-only-log/

45. Append-Only Logs: The Immutable Diary of Data | by komal shehzadi | Medium, accessed June 28, 2025, https://medium.com/@komalshehzadi/append-only-logs-the-immutable-diary-of-data-58c36a871c7c
46. Transparency Logs via Append-Only Authenticated Dictionaries - cse hkust, accessed June 28, 2025, https://cse.hkust.edu.hk/~dipapado/docs/aad.pdf
47. SEMANTIC DRIFT IN ONTOLOGIES - SciTePress, accessed June 28, 2025, https://www.scitepress.org/papers/2010/27888/27888.pdf
48. (PDF) A Framework for Measuring Semantic Drift in Ontologies, accessed June 28, 2025, https://www.researchgate.net/publication/305880437_A_Framework_for_Measuring_Semantic_Drift_in_Ontologies
49. 8 Concept Drift Detection Methods To Use With MI Models - Coralogix, accessed June 28, 2025, https://coralogix.com/ai-blog/concept-drift-8-detection-methods/
50. What is concept drift in ML, and how to detect and address it - Evidently AI, accessed June 28, 2025, https://www.evidentlyai.com/ml-in-production/concept-drift
51. 5 Effective Methods to Detect Concept Drift - Radicalbit MLOps Platform, accessed June 28, 2025, https://radicalbit.ai/resources/blog/detect-concept-drift/
52. AgentLens: Visual Analysis for Agent Behaviors in LLM-based Autonomous Systems - arXiv, accessed June 28, 2025, https://arxiv.org/html/2402.08995v1
53. Visualizing the Evolution of Multi-agent Game-playing Behaviors - Eurographics, accessed June 28, 2025, https://diglib.eg.org/bitstreams/0e6ef74b-b227-4076-bb7f-327b152ebed3/download
54. (PDF) Visualizing Multi-Agent Systems - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/264661166_Visualizing_Multi-Agent_Systems
55. Heatmap visualization | InfluxDB OSS v2 Documentation, accessed June 28, 2025, https://docs.influxdata.com/influxdb/v2/visualize-data/visualization-types/heatmap/
56. Heatmap for Perform Attribute Agreement Study - Support - Minitab, accessed June 28, 2025, https://support.minitab.com/en-us/minitab/help-and-how-to/modules/measurement-system-analysis/attribute-and-inspection-study/perform-attribute-agreement-study/heatmap/
57. How we see time – the evolution and current state of visualizations of temporal data, accessed June 28, 2025, https://www.tandfonline.com/doi/full/10.1080/23729333.2022.2156316
58. Data Visualization: Temporal Tools - Research Guides - Kansas State University, accessed June 28, 2025, https://guides.lib.k-state.edu/c.php?g=181742&p=1196015
59. 9 Data Visualization Techniques for Temporal Mapping That Reveal Hidden Patterns, accessed June 28, 2025, https://www.maplibrary.org/1582/data-visualization-techniques-for-temporal-ma

pping/
60. Multi-Agent AI Success: Performance Metrics and Evaluation Frameworks - Galileo AI, accessed June 28, 2025, https://galileo.ai/blog/success-multi-agent-ai
61. Mastering Multi-Agent Eval Systems in 2025 - Botpress, accessed June 28, 2025, https://botpress.com/blog/multi-agent-evaluation-systems
62. What metrics should I track for semantic search relevance? - Milvus, accessed June 28, 2025, https://milvus.io/ai-quick-reference/what-metrics-should-i-track-for-semantic-search-relevance
63. What are the best tools for simulating multi-agent systems? - Milvus, accessed June 28, 2025, https://milvus.io/ai-quick-reference/what-are-the-best-tools-for-simulating-multiagent-systems
64. Multi-Agent Systems Simulation: Modeling Complex Interactions and Decision-Making Processes - SmythOS, accessed June 28, 2025, https://smythos.com/ai-agents/multi-agent-systems/multi-agent-systems-simulation/
65. How we built our multi-agent research system - Anthropic, accessed June 28, 2025, https://www.anthropic.com/engineering/built-multi-agent-research-system
66. Resilient Consensus Control for Multi-Agent Systems: A Comparative Survey - MDPI, accessed June 28, 2025, https://www.mdpi.com/1424-8220/23/6/2904
67. What is the role of trust in multi-agent systems? - Milvus, accessed June 28, 2025, https://milvus.io/ai-quick-reference/what-is-the-role-of-trust-in-multiagent-systems
68. A survey of the consensus for multi-agent systems, accessed June 28, 2025, https://www.tandfonline.com/doi/abs/10.1080/21642583.2019.1695689
69. Tutorials – AAMAS 2025 Detroit, accessed June 28, 2025, https://aamas2025.org/index.php/conference/program/tutorials/